universität wien

MULTIOMICS DATA SCIENCE - DATA TYPES AND BASIC COMMANDS

INTRODUCTION

- What is computer science?
  - Science of the systematic (computational) processing of information
  - Methodic handling of algorithmically solvable problems

- What is an algorithm?
  - Simplified: a description to solve a problem
  - More precise: formal specific listing of instructions

INTRODUCTION

- Examples for algorithms
  - Cooking recipes
  - Ikea/Lego building instructions

- Algorithms from school or university:
  - Addition of two numbers
  - All possible formulas in math, physics or chemistry
  - SOPs (standard operating procedure)

DATA TYPES

- Elementary data types:
  - Elementary data that are not based on other data types (next slide)
  - Example: integers

- Extended data types:
  - Composition of elementary (and / or non-elementary) data types
  - Example: 2D point (x / y coordinates, consisting of two elementary data types)

ELEMENTARY DATA TYPES

- Booleans: True | False
- Integers: … -2, -1, 0, 1, 2, …
- Real numbers: 0.5, -1.125, 109402.12, …
- Bytes: consisting of 8 bits, smallest adressierable unit in memory of a computer
- Characters: „A“, „2“, „x“, „?“, ...

ELEMENTARY DATA TYPES

- Booleans: True | False
- Integers: … -2, -1, 0, 1, 2, …
- Real numbers: 0.5, -1.125, 109402.12, …
- ~~Bytes: consisting of 8 bits, smallest adressierable unit in memory of a computer~~
- Characters: „A“, „2“, „x“, „?“, ...

INTEGERS (WHOLE NUMBERS)

- In contrast to other programmier languages, Python is able to handle arbitrarily large integers, e.g.:

  542154620715646904712076194203461491479149974230756750910 49

- Integers are stored typically with 64 bits mit 64 Bit, hence the biggest representable number is: $2^{64} - 1 = 18446744073709551615$

- In programming languages such as C++ or Java, large integers $\geq 2^{64}$ must be handled separately

STRINGS

- Sequence of multiple consecutive characters, e.g.: „algorithm“
- Extended data type, since it is a composition
- However, usage of strings is so essential, that they are treated in Python as elementary data types

VARIABLES

- The temporal storage of information and data is done via variables
- All possible data types can be assigned to a variable:

```
age = 32
name = "Jane"
surname = "Doe"
```

- Variable names can be arbitrary as long as they start with a letter and don‘t contain any special characters (except the underscore _) or keywords

VARIABLE ASSIGNMENT

- Different variable assignments

```
age, body_height = 42, 184
weight, voltage = 82.4, -5.2
earthmass_kg, protonmass_kg = 5.9722e24, 1.672621923e-27
light_on = True
car1, car2, amount = "Toyota", 'Audi', 2
telephone = None
```

VARIABLE ASSIGNMENT

- Different variable assignments

```
age, body_height = 42, 184
weight, voltage = 82.4, -5.2
earthmass_kg, protonmass_kg = 5.9722e24, 1.672621923e-27
light_on = True
car1, car2, amount = "Toyota", 'Audi', 2
telephone = None
```

- Strings can be assigned with `"` or `'`, there is no difference
- Several strings can be assigned in one line via comma separation
- `None` is a „void“ value and explicitly not `0`, `False`, or the empty string

KEYWORDS

- In Python, these keywords are reserved for commands and thus cannot be used as variables:

and	del	from	nonlocal	True
as	elif	global	None	try
assert	else	if	not	with
break	except	import	or	while
class	finally	in	pass	yield
continue	False	is	raise	
def	for	lambda	return	

VARIABLE TYPING

- By default, Python uses weak typing of variables
- With the assignment, variables obtain automatically a type and will be treated accordingly
- In contrast to other programming languages:
  - variables don‘t need to be typed strongly, as, e.g., in Java: `int age = 12;`
  - can be overwritten with other types

```
my_variable = 12
my_variable = "Python"
```

ARITHMETIC OPERATIONS

- The known operations / bracketing handling on numbers are:
  - Addition, subtraction: `+`, `-`
  - Multiplication, division: `*`, `/`
  - Modulo (remainder) division: `%`
  - integer division: `//`
  - Potentiation: `**`
  - Brackets: ()

```
n, m = 12.2, 18 + 30
p = m - 1
c= (n - 5)* m    /p
cc    =   m**2 % 12
```

universität wien

ARITHMETIC OPERATIONS

- The known operations / bracketing handling on numbers are:
  - Addition, subtraction: `+`, `-`
  - Multiplication, division: `*`, `/`
  - Modulo (remainder) division: `%`
  - integer division: `//`
  - Potentiation: `**`
  - Brackets: ()

```
n, m = 12.2, 18 + 30
p = m - 1
c= (n - 5)* m    /p
cc    =   m**2 % 12
```

- For Python: Bracket before point before dash calculation!

- Spaces may be used arbitrarily (exceptions are ** or //)

universität wien

COMBINED ASSIGNMENT OPERATORS

- For a shorter notation (and better readability?), arithmetic operators can be combined with the assignment operator

Operation	equivalent to	Operation	equivalent to
`x += a`	`x = x + a`	`x -= a`	`x = x - a`
`x *= a`	`x = x * a`	`x /= a`	`x = x / a`
`x **= a`	`x = x ** a`	`x %= a`	`x = x % a`

universität wien

COMBINED ASSIGNMENT OPERATORS

- For a shorter notation (and better readability?), arithmetic operators can be combined with the assignment operator

Operation	equivalent to	Operation	equivalent to
`x += a`	`x = x + a`	`x -= a`	`x = x - a`
`x *= a`	`x = x * a`	`x /= a`	`x = x / a`
`x **= a`	`x = x ** a`	`x %= a`	`x = x % a`

- These combined operators are interesting, when you want to keep variables and manipulate them

```
a, b = 12, 10
a += 8
a /= b
```

universität wien

FIRST PYTHON SCRIPT

```
print("Hello World!")
num, text = 12, "my first script"
print(num, text)
```

- With the `print()` command, you can display a variable‘s content in a console
- A console is a text-based user interface and can be used for data input and output

COMPARISON OPERATORS

- A comparison operator checks a statement according to its validity and returns either a `True` or `False` value

- These values can also be assigned to a variable:

```
n, m = 12, 18
p = n > 15
q = m == n
```

COMPARISON OPERATORS

- A comparison operator checks a statement according to its validity and returns either a `True` or `False` value

- These values can also be assigned to a variable:

```
n, m = 12, 18
p = n > 15
q = m == n
```

Operator	Comparison	**Operator**	Comparison
<	smaller	**<=**	smaller or equal
>	bigger	**>=**	bigger or equal
==	equal	**!=**	unequal

universität wien

COMPARISON OPERATORS

- A comparison operator checks a statement according to its validity and return either a `True` or `False` value

- These values can also be assigned to a variable:

Caution:

== tests for equality

= Assigns a variable

Operator	Comparison	**Operator**	Comparison
<	smaller	**<=**	smaller or equal
>	bigger	**>=**	bigger or equal
==	equal	**!=**	unequal

universität wien

LOGICAL OPERATORS

- These operators combine boolean values (i.e., `True` or `False`)
- Their return value is again a boolean value

`A`	`B`	`A` **`and`** `B`
False	False	False
False	True	False
True	False	False
True	True	True

`A`	`B`	`A` **`or`** `B`
False	False	False
False	True	True
True	False	True
True	True	True

`A`	**`not`** `A`
False	True
True	False

`A`	`B`	`A ^ B`
False	False	False
False	True	True
True	False	True
True	True	False

universität wien

LOGICAL OPERATORS

```
age, height = 13, 125
may_on_rollercoaster = (age >= 12) and (height >= 120)

may_play_game = (5 <= age) and (age <= 99)
may_play_game2 = 5 <= age <= 99

a, b, c = 2, 3, 5
d = (a != b) or (c >= 2)
e = not d
```

STRINGS

- String (such as „Python“) can be treated as an ordered sequence of single characters
- Ordered means that there is a first char, second char, …
- There are operations, that you can apply on strings to manipulate these
- Many operations on strings are already implemented in Python
- One of the most basic operations is the equality comparison

```
a, b = "string", "sting"
c, d = a == b, a != b
```

STRINGS AND THEIR FUNCTIONS

- To execute manipulative operations on strings, you have to call the according function on the respective variable, which looks like this:

  variable.functionname()

- Counts the number of a pattern in the text:

```
s = "string in internet"
l = s.count("in")      # returns 3
```

- Replaces all upper characters with lower:

```
s = s.lower()      # remains unchanged
```

- Replaces all lower characters with upper:

```
s = s.upper()      # returns "STRING IN INTERNET"
```

STRINGS AND THEIR FUNCTIONS

- Replaces all occurrences of a pattern in a text with another:

```
s = "string in internet"
l = s.replace("in", "out")    # "stroutg out outternet"
```

- Searches a pattern in a text and returns its first position, otherwise -1:

```
pos = s.find("in")     # returns 3
```

- Trims a pattern at the beginning and end of a string:

```
s = "...abracadabra..."
s = s.strip(".")       # returns "abracadabra"
```

INDEXING AND SLICING

- Since a string is ordered, one can access its elements
- This type of indexing is realized with [] operators
- Caution: ordered data structures (e.g., strings) always start with index 0

```
s = "hacoona matata"
c = s[0]        # returns h
c2 = s[2]       # returns c
```

INDEXING AND SLICING

- It is very neat to access the last elements of a string by using negative indexes

```
s = "hacoona matata"
c = s[-1]       # returns "a"
c2 = s[-4]      # returns "t"
```

- It is of course not possible to access the 20th character of a string with on 10 characters

- In this case, Python throws an exception and interrupts the script completely

INDEXING AND SLICING

- Substrings can also be extracted with little effort by using the [] operators
- A start index and an end index have to be provided, separated by a :
- The end index is always exclusive

```
s = "simsalabim"
c = s[1:4]      # returns "ims"
c2 = s[3:8]     # returns "salab"
```

INDEXING AND SLICING

- To obtain prefixes and suffixes, one can either provide the first / last index explicitly, or just leave it out

```
s = "Yabadabadoo"
pre1 = s[0:3]      # returns "Yab"
pre2 = s[:5]       # returns "Yabab", too
suf1 = s[8:11]     # returns "doo"
suf2 = s[8:]       # returns "doo", too
suf3 = s[-3:]      # returns "doo", too
```

INDEXING AND SLICING

- Additionally, one can define a step value, which is set to 1 by default

- The syntax is as follows [start index : end index : step value]

```
s = "Yabadabadoo"
step = s[::2]          # returns "Ybdbdo"
step2 = s[1:9:3]       # returns "ada"
```

- And you can even go backwards:

```
back = s[3::-1]        # returns "abaY"
back = s[8:3:-2]       # returns "dbd"
```

INDEXING AND SLICING

- Additionally, one can define a step value, which is set to 1 by default
- The syntax is as follows [start index : end index : step value]

Caution:
Start index is always inclusive
End index is always exclusive

- And you can even go backwards:

```
back = s[3::-1]        # returns "abaY"
back = s[8:3:-2]       # returns "dbd"
```

COMMENTS

- Comments serve in the code for the understanding of the developer and are explicitly not executed by the interpreter / processor

- Comments occur in all programming languages, in Python single-row commpents are initiated by #

```
# Now we define funny variables
a = "my Text"           # a will be used later
b, c = 10, 100          # b und c define foo and bar
```

- Everything that comes after a #, will be ignored by the interpreter

COMMENTS

- Multi-line blocks of commentars can be achieved in two ways
- Of course, with each line starting with a #
- A more elegant solution is to usage the three-fold """ at the beginning and end of a comment

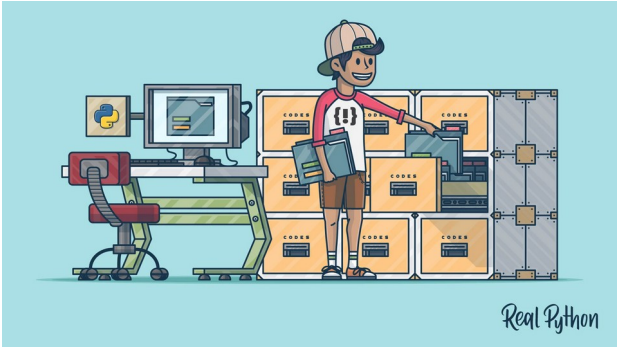
```
"""
The is our deprecated code, it used to be useful
a = "my Text"
b, c = 10, 100     # b und c are numbers
"""
```

COMMENTS

- Comments serve the following funktions
  - Assistance / explanation of the developer for certain code areas
  - Commenting of (temporarily unnecessary) code
  - Documentation of functions
- Several frameworks read in the complete source code and create automatically a documentation about the programmed functions based on the provided comments

DOCUMENATION

- *https://docs.python.org/3/*
- Google / DuckDuckGo
- Python Crash Course, 2nd Edition; ISBN: 978-1593279288



https://realpython.com/documenting-python-code/